

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering Academic Year 2025-26

Rubrics for Lab Experiments

**Class : B.E. Computer
Engineering**

Subject Name :DC Lab

Semester : VIII

Subject Code : CSL801

Practical No:	7
Title:	Design and implement a distributed file system that allows multiple nodes to share and access files in a distributed environment.
Date of Performance:	17/11/25
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation:

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Aim:

Design and implement a distributed file system that allows multiple nodes to share and access files in a distributed environment.

Request.java

```
import java.io.Serializable;
import java.util.Map;

public class Request implements Serializable {
    public enum Type {
        REGISTER_CHUNKSERVER, HEARTBEAT,
        CREATE_FILE, GET_FILE_INFO,
        STORE_CHUNK, FETCH_CHUNK, DELETE_CHUNK,
        SHUTDOWN
    }
    public Type type;
    public Map<String, Object> data;
    public Request(Type type, Map<String, Object> data) {
        this.type = type; this.data = data;
    }
}
```

Response.java

```
import java.io.Serializable;
import java.util.Map;

public class Response implements Serializable {
    public boolean ok;
    public String error;
    public Map<String, Object> data;
    public Response(boolean ok, String error, Map<String, Object> data) {
        this.ok = ok; this.error = error; this.data = data;
    }
    public static Response success(Map<String, Object> data) {
        return new Response(true, null, data);
    }
    public static Response fail(String err) {
        return new Response(false, err, null);
    }
}
```

ChunkInfo.java

```
import java.io.Serializable;
import java.util.List;

public class ChunkInfo implements Serializable {
    public String chunkId;
    public List<String> locations; // host:port strings
    public ChunkInfo(String chunkId, List<String> locations) {
        this.chunkId = chunkId;
        this.locations = locations;
    }
}
```

MetadataServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class MetadataServer {
    private final int port;
    private final Map<String, List<String>> fileToChunks = new ConcurrentHashMap<>();
    private final Map<String, List<String>> chunkToLocations = new ConcurrentHashMap<>();
    private final Set<String> chunkServers = ConcurrentHashMap.newKeySet();
    private final int replicationFactor = 2; // configurable
    private final Map<String, Long> lastHeartbeat = new ConcurrentHashMap<>();
    private final static long HEARTBEAT_TIMEOUT_MS = 20000;

    public MetadataServer(int port) {
        this.port = port;
    }

    public void start() throws Exception {
        // background: monitor heartbeats

        Executors.newSingleThreadScheduledExecutor().scheduleAtFixedRate(this::cleanupDeadC
            hunkServers, 5, 5, TimeUnit.SECONDS);

        ServerSocket ss = new ServerSocket(port);
        System.out.println("MetadataServer listening on " + port);
        while (true) {
            final Socket s = ss.accept();
            Executors.newCachedThreadPool().execute(() -> handleConnection(s));
        }
    }
}
```

```

}

private void handleConnection(Socket s) {
    try(ObjectInputStream in = new ObjectInputStream(s.getInputStream());
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream())) {
        Request req = (Request) in.readObject();
        Response resp = processRequest(req);
        out.writeObject(resp);
        out.flush();
    } catch (Exception e) {
        System.err.println("Metadata connection error: " + e);
    } finally {
        try { s.close(); } catch(Exception ignored) {}
    }
}

private synchronized Response processRequest(Request req) {
    try {
        switch (req.type) {
            case REGISTER_CHUNKSERVER: {
                String addr = (String) req.data.get("address"); // host:port
                chunkServers.add(addr);
                lastHeartbeat.put(addr, System.currentTimeMillis());
                System.out.println("Registered chunkserver " + addr);
                return Response.success(Map.of("message", "registered"));
            }
            case HEARTBEAT: {
                String addr = (String) req.data.get("address");
                lastHeartbeat.put(addr, System.currentTimeMillis());
                return Response.success(Map.of("message", "heartbeat"));
            }
            case CREATE_FILE: {
                String filename = (String) req.data.get("filename");
                long filesize = (Long) req.data.get("size");
                int chunkSize = (Integer) req.data.get("chunkSize");
                int numChunks = (int) Math.max(1, Math.ceil((double)filesize / chunkSize));
                List<String> chunkIds = new ArrayList<>();
                List<Map<String, Object>> allocation = new ArrayList<>();
                List<String> available = new ArrayList<>(chunkServers);
                if (available.isEmpty()) return Response.fail("No chunkservers available");
                for (int i=0; i<numChunks; i++) {
                    String chunkId = filename + "_chunk_" + i + "_" +
                        UUID.randomUUID().toString();
                    chunkIds.add(chunkId);
                    // choose up to replicationFactor distinct servers (round-robin)
                    List<String> chosen = new ArrayList<>();
                    for (int r=0; r<replicationFactor && !available.isEmpty(); r++) {
                        String pick = available.get((i + r) % available.size());

```

```

        if (!chosen.contains(pick)) chosen.add(pick);
    }
    chunkToLocations.put(chunkId, new ArrayList<>(chosen));
    allocation.add(Map.of("chunkId", chunkId, "locations", chosen));
}
fileToChunks.put(filename, chunkIds);
Map<String, Object> data = new HashMap<>();
data.put("chunks", allocation);
System.out.println("Created file metadata for " + filename + " chunks=" +
chunkIds.size());
return Response.success(data);
}
case GET_FILE_INFO: {
    String filename = (String) req.data.get("filename");
    List<String> chunks = fileToChunks.get(filename);
    if (chunks==null) return Response.fail("File not found");
    List<Map<String, Object>> infos = new ArrayList<>();
    for (String cid: chunks) {
        List<String> locs = chunkToLocations.getDefault(cid, List.of());
        infos.add(Map.of("chunkId", cid, "locations", locs));
    }
    return Response.success(Map.of("chunks", infos));
}
case SHUTDOWN: {
    System.out.println("Shutdown requested.");
    return Response.success(Map.of("message", "bye"));
}
default:
    return Response.fail("Unsupported request to metadata server");
}
} catch (Exception e) {
    e.printStackTrace();
    return Response.fail("Exception " + e.getMessage());
}
}

```

```

private void cleanupDeadChunkServers() {
    long now = System.currentTimeMillis();
    List<String> dead = new ArrayList<>();
    for (var entry: lastHeartbeat.entrySet()) {
        if (now - entry.getValue() > HEARTBEAT_TIMEOUT_MS) dead.add(entry.getKey());
    }
    for (String addr: dead) {
        System.out.println("Chunkserver timed out: " + addr);
        chunkServers.remove(addr);
        lastHeartbeat.remove(addr);
        // remove from chunkToLocations entries
        for (var kv: chunkToLocations.entrySet()) {

```

```

        kv.getValue().remove(addr);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length < 1) { System.out.println("Usage: java MetadataServer <port>"); return; }
    int port = Integer.parseInt(args[0]);
    new MetadataServer(port).start();
}
}

```

ChunkServer.java

```

import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class ChunkServer {
    private final String metadataHost;
    private final int metadataPort;
    private final int port;
    private final File storageDir;
    private final String address; // host:port
    private final ScheduledExecutorService scheduler =
        Executors.newScheduledThreadPool(1);

    public ChunkServer(String metadataHost, int metadataPort, int port, String storageDir) {
        this.metadataHost = metadataHost; this.metadataPort = metadataPort; this.port = port;
        this.storageDir = new File(storageDir);
        if (!this.storageDir.exists()) this.storageDir.mkdirs();
        this.address = "localhost:" + port;
    }

    public void start() throws Exception {
        registerWithMetadata();
        scheduler.scheduleAtFixedRate(this::sendHeartbeat, 5, 5, TimeUnit.SECONDS);

        ServerSocket ss = new ServerSocket(port);
        System.out.println("ChunkServer listening on " + port + ", storage=" +
            storageDir.getAbsolutePath());
        while (true) {
            Socket s = ss.accept();
            Executors.newCachedThreadPool().execute(() -> handleConn(s));
        }
    }
}

```

```
}
```

```
private void registerWithMetadata() {  
    try (Socket s = new Socket(metadataHost, metadataPort);  
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());  
        ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {  
        Map<String, Object> data = Map.of("address", address);  
        out.writeObject(new Request(Request.Type.REGISTER_CHUNKSERVER, data));  
        out.flush();  
        Response r = (Response) in.readObject();  
        if (!r.ok) System.err.println("Register failed: " + r.error);  
    } catch (Exception e) {  
        System.err.println("Register exception: " + e.getMessage());  
    }  
}
```

```
private void sendHeartbeat() {  
    try (Socket s = new Socket(metadataHost, metadataPort);  
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());  
        ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {  
        out.writeObject(new Request(Request.Type.HEARTBEAT, Map.of("address",  
address)));  
        out.flush();  
        // ignore response  
        in.readObject();  
    } catch (Exception e) {  
        System.err.println("Heartbeat failed: " + e.getMessage());  
    }  
}
```

```
private void handleConn(Socket s) {  
    try (ObjectInputStream in = new ObjectInputStream(s.getInputStream());  
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream())) {  
        Request req = (Request) in.readObject();  
        switch (req.type) {  
            case STORE_CHUNK: {  
                String chunkId = (String) req.data.get("chunkId");  
                int len = (Integer) req.data.get("length");  
                File f = new File(storageDir, chunkId);  
                try (FileOutputStream fos = new FileOutputStream(f)) {  
                    byte[] buf = new byte[8192];  
                    int remaining = len;  
                    InputStream is = s.getInputStream();  
                    while (remaining > 0) {  
                        int toRead = Math.min(buf.length, remaining);  
                        int read = is.read(buf, 0, toRead);  
                        if (read == -1) break;  
                        fos.write(buf, 0, read);  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        remaining -= read;
    }
}
out.writeObject(Response.success(Map.of("message", "stored")));
out.flush();
break;
}
case FETCH_CHUNK: {
    String chunkId = (String) req.data.get("chunkId");
    File f = new File(storageDir, chunkId);
    if (!f.exists()) { out.writeObject(Response.fail("chunk not found")); break; }
    long len = f.length();
    out.writeObject(Response.success(Map.of("length", len)));
    out.flush();
    // then send bytes
    try (FileInputStream fis = new FileInputStream(f);
        OutputStream os = s.getOutputStream()) {
        byte[] buf = new byte[8192];
        int r;
        while ((r = fis.read(buf)) != -1) os.write(buf, 0, r);
        os.flush();
    }
    break;
}
case DELETE_CHUNK: {
    String chunkId = (String) req.data.get("chunkId");
    File f = new File(storageDir, chunkId);
    boolean ok = f.delete();
    out.writeObject(ok ? Response.success(Map.of("deleted", true)) :
Response.fail("delete failed"));
    break;
}
default:
    out.writeObject(Response.fail("unsupported op for chunkserver"));
}
} catch (Exception e) {
    System.err.println("ChunkServer conn error: " + e.getMessage());
} finally {
    try { s.close(); } catch (Exception ignored) {}
}
}

public static void main(String[] args) throws Exception {
    if (args.length < 4) {
        System.out.println("Usage: java ChunkServer <metadataHost> <metadataPort>
<port> <storageDir>");
        return;
    }
}

```



```

        String mh = args[0];
        int mp = Integer.parseInt(args[1]);
        int port = Integer.parseInt(args[2]);
        String dir = args[3];
        new ChunkServer(mh, mp, port, dir).start();
    }
}

```

Client.java

```

import java.io.*;
import java.net.*;
import java.util.*;

public class Client {
    private final String metadataHost;
    private final int metadataPort;
    private final int chunkSize = 1024 * 1024; // 1MB

    public Client(String metadataHost, int metadataPort) {
        this.metadataHost = metadataHost; this.metadataPort = metadataPort;
    }

    public void upload(String localPath, String remoteName) throws Exception {
        File f = new File(localPath);
        if (!f.exists()) throw new FileNotFoundException(localPath);
        long size = f.length();
        // ask metadata to create file
        Response r = sendMetadataRequest(new Request(Request.Type.CREATE_FILE,
            Map.of("filename", remoteName, "size", size, "chunkSize", chunkSize)));
        if (!r.ok) throw new RuntimeException("Create failed: " + r.error);
        @SuppressWarnings("unchecked")
        List<Map<String, Object>> chunks = (List<Map<String, Object>>) r.data.get("chunks");
        // send each chunk to its allocated chunkservers (first one is primary for our simple
        approach)
        try (FileInputStream fis = new FileInputStream(f)) {
            byte[] buffer = new byte[8192];
            for (Map<String, Object> cinfo : chunks) {
                String chunkId = (String) cinfo.get("chunkId");
                @SuppressWarnings("unchecked")
                List<String> locs = (List<String>) cinfo.get("locations");
                // read chunk bytes to memory (could stream but simpler)
                ByteArrayOutputStream baos = new ByteArrayOutputStream();
                int toRead = chunkSize;
                while (toRead > 0) {
                    int read = fis.read(buffer, 0, Math.min(buffer.length, toRead));

```

```

        if (read <= 0) break;
        baos.write(buffer, 0, read);
        toRead -= read;
    }
    byte[] chunkBytes = baos.toByteArray();
    // send copy to every location (replication)
    for (String loc : locs) {
        String[] hp = loc.split(":");
        String host = hp[0];
        int port = Integer.parseInt(hp[1]);
        System.out.println("Uploading chunk " + chunkId + " to " + loc + " size=" +
chunkBytes.length);
        try (Socket s = new Socket(host, port);
            ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
            ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {
            out.writeObject(new Request(Request.Type.STORE_CHUNK,
Map.of("chunkId", chunkId, "length", chunkBytes.length)));
            out.flush();
            // write bytes directly to socket (after header)
            OutputStream os = s.getOutputStream();
            os.write(chunkBytes);
            os.flush();
            Response rr = (Response) in.readObject();
            if (!rr.ok) System.err.println("Store chunk failed on " + loc + ": " + rr.error);
        } catch (Exception e) {
            System.err.println("Error uploading chunk to " + loc + ": " + e.getMessage());
        }
    }
}
}
}
}
System.out.println("Upload complete for " + remoteName);
}

```

```

public void download(String remoteName, String outputPath) throws Exception {
    Response r = sendMetadataRequest(new Request(Request.Type.GET_FILE_INFO,
Map.of("filename", remoteName)));
    if (!r.ok) throw new RuntimeException("Get info failed: " + r.error);
    @SuppressWarnings("unchecked")
    List<Map<String, Object>> chunks = (List<Map<String, Object>>) r.data.get("chunks");
    try (FileOutputStream fos = new FileOutputStream(outputPath)) {
        for (Map<String, Object> cinfo : chunks) {
            String chunkId = (String) cinfo.get("chunkId");
            @SuppressWarnings("unchecked")
            List<String> locs = (List<String>) cinfo.get("locations");
            boolean fetched = false;
            for (String loc : locs) {
                try {
                    String[] hp = loc.split(":");

```

```

        String host = hp[0]; int port = Integer.parseInt(hp[1]);
        try (Socket s = new Socket(host, port);
            ObjectOutputStream out = new
ObjectOutputStream(s.getOutputStream());
            ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {
            out.writeObject(new Request(Request.Type.FETCH_CHUNK,
Map.of("chunkId", chunkId)));
            out.flush();
            Response rr = (Response) in.readObject();
            if (!rr.ok) { System.err.println("fetch failed: " + rr.error); continue; }
            int len = ((Long) rr.data.get("length")).intValue();
            InputStream is = s.getInputStream();
            byte[] buf = new byte[8192];
            int remaining = len;
            while (remaining > 0) {
                int read = is.read(buf, 0, Math.min(buf.length, remaining));
                if (read == -1) break;
                fos.write(buf, 0, read);
                remaining -= read;
            }
            fetched = true; break;
        } catch (Exception e) {
            System.err.println("Attempt failed for loc " + loc + " : " + e.getMessage());
        }
    }
    if (!fetched) throw new RuntimeException("Failed to fetch chunk " + chunkId);
}
}
System.out.println("Downloaded file to " + outPath);
}

private Response sendMetadataRequest(Request req) throws Exception {
    try (Socket s = new Socket(metadataHost, metadataPort);
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
        ObjectInputStream in = new ObjectInputStream(s.getInputStream())) {
        out.writeObject(req);
        out.flush();
        return (Response) in.readObject();
    }
}

public static void main(String[] args) throws Exception {
    if (args.length < 1) {
        System.out.println("Usage: java Client <upload|download> <metadataHost>
<metadataPort> ...");
        return;
    }
}

```

```

String op = args[0];
if (op.equals("upload")) {
    if (args.length != 5) { System.out.println("upload usage: java Client upload
<metadataHost> <metadataPort> <localPath> <remoteName>"); return; }
    String mh = args[1]; int mp = Integer.parseInt(args[2]); String localPath = args[3];
String remote = args[4];
    new Client(mh, mp).upload(localPath, remote);
} else if (op.equals("download")) {
    if (args.length != 5) { System.out.println("download usage: java Client download
<metadataHost> <metadataPort> <remoteName> <outPath>"); return; }
    String mh = args[1]; int mp = Integer.parseInt(args[2]); String remote = args[3]; String
outPath = args[4];
    new Client(mh, mp).download(remote, outPath);
} else {
    System.out.println("Unknown op: " + op);
}
}
}
}

```

```

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7
● > javac *.java

```

```

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7
za > java MetadataServer 9000
MetadataServer listening on 9000
[]

```

Welcome to Hallownest, little ghost...

```

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7
○ > java ChunkServer localhost 9000 10001 data1
ChunkServer listening on 10001, storage = C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7\data1
[]

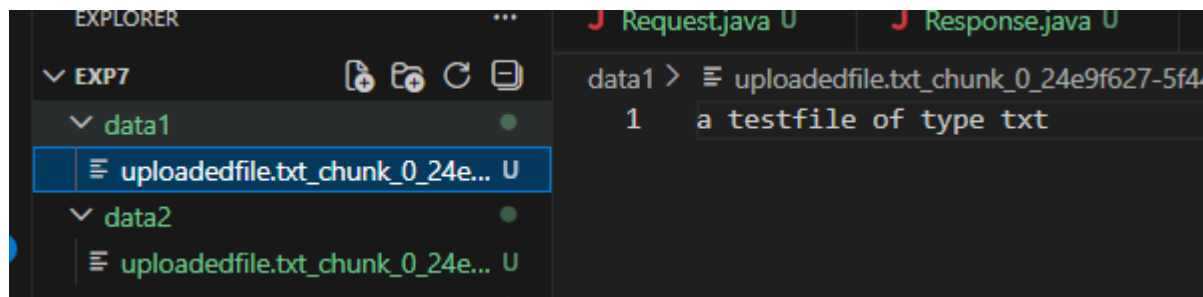
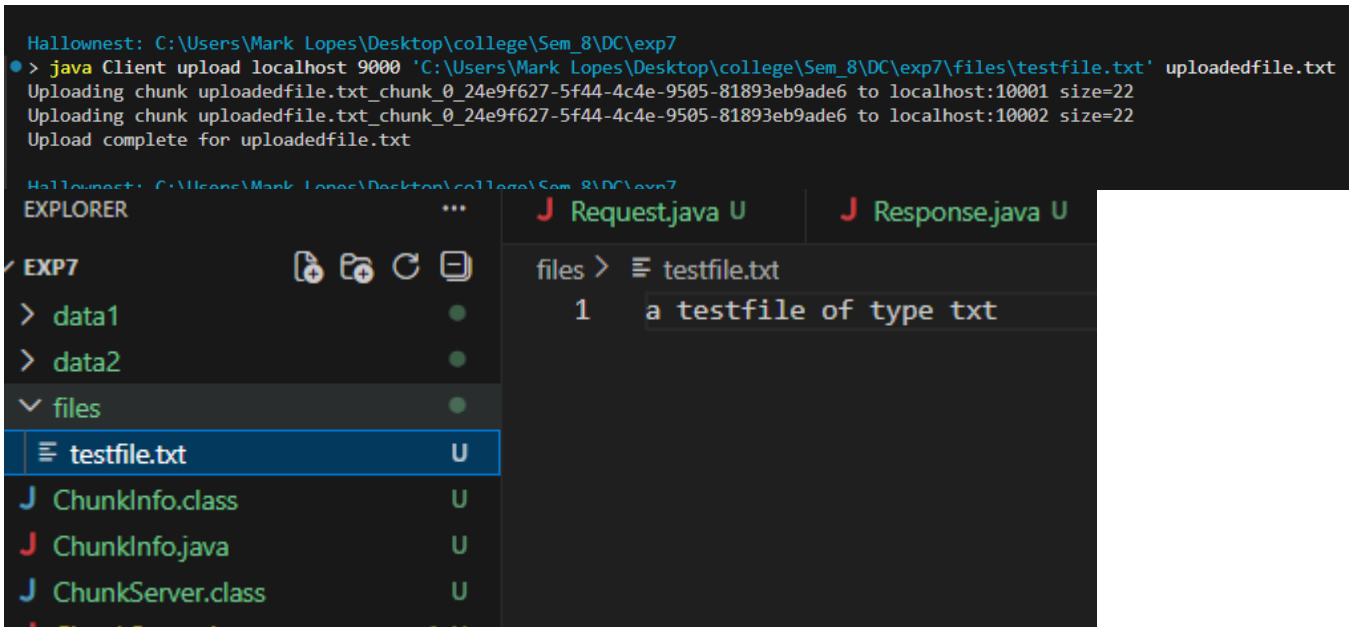
```

Welcome to Hallownest, little ghost...

```

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7
○ > java ChunkServer localhost 9000 10002 data2
ChunkServer listening on 10002, storage = C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7\data2
[]

```



```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7
> java Client download localhost 9000 uploadedfile.txt 'C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7\files\downloadedfile.txt'
Downloaded file to C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7\files\downloadedfile.txt
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp7
```

EXPLORER

EXP7

data1

uploadedfile.txt_chunk_0_24e... U

data2

uploadedfile.txt_chunk_0_24e... U

files

downloadedfile.txt U

testfile.txt U

ChunkInfo.class U

ChunkInfo.java U

Request.java U

Response.java U

Chu

files >

downloadedfile.txt

1 a testfile of type txt